

空间相机通信协议V1.0

1、概述

本文档定义 active_app 基于 Zenoh (<https://zenoh.io>) 的数据传输协议。利用 Zenoh 原生的 pub/sub 和 queryable 机制进行数据分发与指令交互，消息 payload 采用 Protobuf 编码。

2、架构

代码块

```
1  数据通道（设备 → 订阅端）：Zenoh pub/sub
2      设备 put(key_expr, payload)
3      订阅端 subscribe(key_expr)
4
5  指令通道（请求端 → 设备）：Zenoh queryable
6      设备 declare_queryable(key_expr)
7      请求端 get(key_expr, payload) → reply
```

设备端以 **peer** 模式运行，支持：

- 局域网内直接 peer-to-peer 通信（多播自动发现）
- 通过 Zenoh router 中转（跨网段场景）

3、C 绑定

使用 **zenoh-c**（zenoh-pico 作为备选方案用于资源受限场景）。

4、设备标识

每台设备通过 SN 号区分（从 `/mnt/system/factory-data/Lixel.yaml` 的 `device_sn` 字段读取）。SN 作为 key expression 的前缀，用于多设备区分。

5、Key Expression 定义

格式： `active/{sn}/{data_type}`

数据通道（pub/sub）

Key Expression	Protobuf 消息类型	频率	说明
active/{sn}/pointcloud	LidarPointMsgArray	5 Hz	三维点坐标+强度+环号
active/{sn}/image	ImageMsgArray	10 Hz	编码后图像（JPEG）
active/{sn}/imu	ImuMsgArray	50 Hz	加速度+角速度
active/{sn}/config_file	FileMsgArray	按需	配置文件传输

订阅方式

代码块

```
1 // 订阅特定设备全部数据
2 session.declare_subscriber("active/LX2601F10001/**")
3
4 // 订阅特定设备 IMU
5 session.declare_subscriber("active/LX2601F10001/imu")
6
7 // 订阅所有设备全部数据
8 session.declare_subscriber("active/**")
9
10 // 订阅所有设备点云
11 session.declare_subscriber("active/*/pointcloud")
```

指令通道（queryable）

Key Expression	方向	说明
active/{sn}/cmd/{cmd_name}	请求端 → 设备	指令请求

设备端声明 queryable: active/{sn}/cmd/*

请求端通过 get() 发送指令，设备端在 queryable 回调中处理并 reply()。

6、Protobuf 消息定义

消息定义文件：

代码块

```
1  syntax = "proto3";
2  package active_msgs;
3
4  message Header {
5      uint32 seq = 1;    // 帧序号
6      sint64 stamp = 2; // 帧时间戳
7      sint64 scaler = 3; // 缩放系数(正数:放大倍数,负数:缩小倍数,0:禁用缩放)
8  }
9
10 message Error {
11     uint32 code = 1; // 错误码
12     string desc = 2; // 描述符
13 }
14
15 message ImuMsg {
16     Header header = 1; // scaler = 10000
17     sint64 ax = 2;      // val = 原始值 * header.scaler
18     sint64 ay = 3;
19     sint64 az = 4;
20     sint64 gx = 5;
21     sint64 gy = 6;
22     sint64 gz = 7;
23 }
24
25 message LidarPoint {
26     sint64 x = 1;      // val = 原始值 * header.scaler
27     sint64 y = 2;
28     sint64 z = 3;
29     uint32 rgbi = 4; // R+G+B+Intensity
30     uint32 ring = 5;
31     sint32 offset = 6; // 点云时间戳偏移值
32 }
33
34 message LidarPointMsg {
35     Header header = 1; // scaler = 1000
36     repeated LidarPoint points = 2;
37 }
38
39 enum ImageFormat {
40     ImageFormatRaw = 0;
41     ImageFormatRgb = 1;
42     ImageFormatJpeg = 2;
43     ImageFormatH264 = 3;
44 }
45
46 message ImageMsg {
47     Header header = 1;
```

```

48     ImageFormat format = 2;
49     uint32 width = 3;
50     uint32 height = 4;
51     bytes data = 5;
52 }
53
54 message FileMsg {
55     Header header = 1;
56     string path = 2;
57     bytes data = 3;
58 }
59
60 //-----
61 // topic: active/{sn}/imu
62 message ImuMsgArray {
63     repeated ImuMsg array = 1;
64 }
65
66 // topic: active/{sn}/pointcloud
67 message LidarPointMsgArray {
68     repeated LidarPointMsg array = 1;
69 }
70
71 // topic: active/{sn}/config_file
72 message FileMsgArray {
73     repeated FileMsg array = 1;
74 }
75
76 // topic: active/{sn}/image
77 message ImageMsgArray {
78     repeated ImageMsg array = 1;
79 }
80
81 // queryable: active/{sn}/cmd/setting
82 message SettingRequest {
83     Header header = 1;
84     sint32 imu_fps = 2;
85     sint32 lidar_fps = 3;
86     sint32 image_fps = 4;
87     sint32 image_format = 5; // 0:无效 1:H264 2:JPEG
88     sint32 h264_gop = 6;
89     sint32 h264_bitrate = 7;
90 }
91
92 message SettingResponse {
93     Header header = 1;
94     sint32 imu_fps = 2;

```

```

95     sint32 lidar_fps = 3;
96     sint32 image_fps = 4;
97     sint32 image_format = 5;
98     sint32 h264_gop = 6;
99     sint32 h264_bitrate = 7;
100    string parameter = 8;
101    Error error = 15;
102 }
103
104 // queryable: active/{sn}/cmd/reboot
105 message RebootRequest {
106     Header header = 1;
107 }
108
109 message RebootResponse {
110     Header header = 1;
111     Error error = 15;
112 }
113

```

公共帧头 Header

代码块

```

1  message Header {
2      uint32 seq = 1;    // 帧序号
3      sint64 stamp = 2;  // 帧时间戳
4      sint64 scaler = 3; // 缩放系数(正数:放大倍数,负数:缩小倍数,0:禁用缩放)
5  }
6

```

所有消息均包含 `Header`，其中 `scaler` 字段用于整数化浮点值：实际值 = 整数值 / scaler。

错误消息

代码块

```

1  message Error {
2      uint32 code = 1; // 错误码 (0=成功, 1=解析失败, 2=保存失败)
3      string desc = 2; // 错误描述
4  }
5

```

IMU 消息

Topic: `active/{sn}/imu`

代码块

```
1  message ImuMsg {
2      Header header = 1; // scaler = 10000
3      sint64 ax = 2;      // 加速度 X, 实际值 = ax / 10000
4      sint64 ay = 3;      // 加速度 Y
5      sint64 az = 4;      // 加速度 Z
6      sint64 gx = 5;      // 角速度 X
7      sint64 gy = 6;      // 角速度 Y
8      sint64 gz = 7;      // 角速度 Z
9  }
10
11 message ImuMsgArray {
12     repeated ImuMsg array = 1;
13 }
14
```

点云消息

Topic: `active/{sn}/pointcloud`

代码块

```
1  message LidarPoint {
2      sint64 x = 1;      // 坐标 X, 实际值 = x / 1000
3      sint64 y = 2;      // 坐标 Y
4      sint64 z = 3;      // 坐标 Z
5      uint32 rgbi = 4;    // R+G+B+Intensity (各8位打包)
6      uint32 ring = 5;    // 环号
7      sint32 offset = 6;  // 时间戳偏移 (时间戳 = header.stamp + offset)
8  }
9
10 message LidarPointMsg {
11     Header header = 1;    // scaler = 1000
12     repeated LidarPoint points = 2;
13 }
14
15 message LidarPointMsgArray {
16     repeated LidarPointMsg array = 1;
17 }
18
```

图像消息

Topic: active/{sn}/image

代码块

```
1  enum ImageFormat {
2      ImageFormatRaw = 0;
3      ImageFormatRgb = 1;
4      ImageFormatJpeg = 2;
5      ImageFormatH264 = 3;
6  }
7
8  message ImageMsg {
9      Header header = 1;
10     ImageFormat format = 2; // 编码格式
11     uint32 width = 3;      // 图像宽度
12     uint32 height = 4;     // 图像高度
13     bytes data = 5;        // 图像数据
14 }
15
16 message ImageMsgArray {
17     repeated ImageMsg array = 1;
18 }
19
```

文件消息

Topic: active/{sn}/config_file

代码块

```
1  message FileMsg {
2      Header header = 1;
3      string path = 2; // 文件路径: 如标定文件"device_calib_info"
4      bytes data = 3;  // 文件内容
5  }
6
7  message FileMsgArray {
8      repeated FileMsg array = 1;
9  }
10
```

7、数据编码约定

缩放系数 (scaler)

为避免浮点数精度损失和传输开销, 采用整数化传输:

消息类型	scaler	说明
ImuMsg	10000	6位有效数字精度
LidarPointMsg	1000	毫米精度

接收端还原公式：`实际值 = 整数值 / header.scaler`

批量消息 (Array)

每个 topic 的 payload 均为对应的 `XxxMsgArray` 类型。即使单帧数据也封装为数组（array 长度为 1），保持接口一致性。

8、指令通道

请求格式

请求端通过 `get()` 发送指令：

- **Key expression:** `active/{sn}/cmd/{cmd_name}`
- **Payload:** Protobuf 编码的指令参数（格式由 `cmd_name` 决定）

应答格式

设备端通过 `reply()` 返回结果：

- **Payload:** Protobuf 编码的应答数据（格式由 `cmd_name` 决定）

已定义指令

cmd_name	请求消息	应答消息	说明
<code>setting</code>	<code>SettingRequest</code>	<code>SettingResponse</code>	修改传感器参数（帧率、编码格式等），配置持久化到
<code>reboot</code>	<code>RebootRequest</code>	<code>RebootResponse</code>	软重启设备，设备在回复后延迟 500ms 执行 reboot

setting 指令

请求消息：

代码块

```
1 message SettingRequest {
```



```

2   Header header = 1;
3   sint32 imu_fps = 2;           // IMU 帧率, >0 时生效
4   sint32 lidar_fps = 3;        // 点云帧率 (配置值), >0 时生效
5   sint32 image_fps = 4;        // 图像帧率, >0 时生效
6   sint32 image_format = 5;     // 0:H264 1:JPEG, >0 时生效
7   sint32 h264_gop = 6;         // H264 GOP 大小, >0 时生效
8   sint32 h264_bitrate = 7;     // H264 码率(Mbps), >0 时生效
9 }
10

```

应答消息:

代码块

```

1  message SettingResponse {
2      Header header = 1;
3      sint32 imu_fps = 2;
4      sint32 lidar_fps = 3;
5      sint32 image_fps = 4;
6      sint32 image_format = 5;
7      sint32 h264_gop = 6;
8      sint32 h264_bitrate = 7;
9      string parameter = 8;     // 预留扩展参数
10     Error error = 15;         // 错误信息 (code=0 表示成功)
11 }
12

```

 仅非零字段会被更新, 重启后生效。应答中返回当前所有参数的最新值。

reboot 指令

请求消息:

代码块

```

1  message RebootRequest {
2      Header header = 1;
3  }
4

```

应答消息:

代码块

```
1 message RebootResponse {
2     Header header = 1;
3     Error error = 15; // 错误信息 (code=0 表示成功)
4 }
5
```



设备在发送应答后延迟 500ms 执行系统重启，确保应答数据发送完成。

错误码定义

code	说明
0	成功
1	请求解析失败
2	配置保存失败

9、配置

config.json 配置节

代码块

```
1 {
2     "zenoh": {
3         "mode": "peer",
4         "connect": [],
5         "listen": [],
6         "scouting": {
7             "multicast": {
8                 "enabled": true
9             }
10        }
11    }
12 }
```

字段	类型	默认值	说明
mode	string	"peer"	Zenoh 会话模式：peer 或 client

<code>connect</code>	<code>string[]</code>	<code>[]</code>	主动连接的 router/peer 地址，如 <code>["tcp/192.168.1.1:7447"]</code>
<code>listen</code>	<code>string[]</code>	<code>[]</code>	监听地址（peer 模式下可为空，依赖自动发现）
<code>scouting.multicast.enabled</code>	<code>bool</code>	<code>true</code>	是否启用多播自动发现

10、QoS 配置

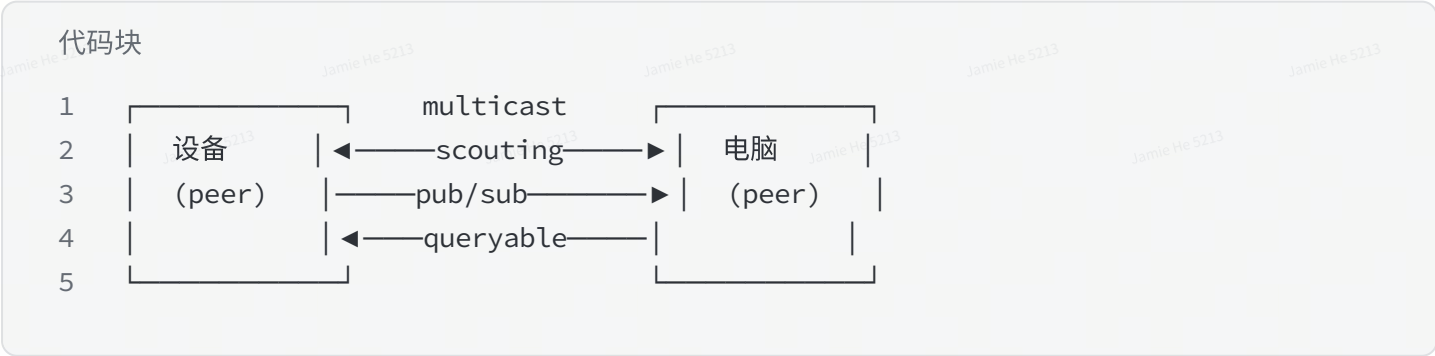
Zenoh 原生支持 QoS，各数据类型推荐配置：

数据类型	Congestion Control	Priority	Reliability
<code>pointcloud</code>	Drop	Data (5)	BestEffort
<code>image</code>	Drop	Data (5)	BestEffort
<code>imu</code>	Drop	RealTime (6)	BestEffort
<code>config_file</code>	Block	Data (5)	Reliable
<code>cmd</code>	Block	Data (5)	Reliable

- **Drop:** 发送缓冲区满时丢弃旧数据（适合高频传感器数据）
- **Block:** 发送缓冲区满时阻塞等待（适合指令和文件传输）

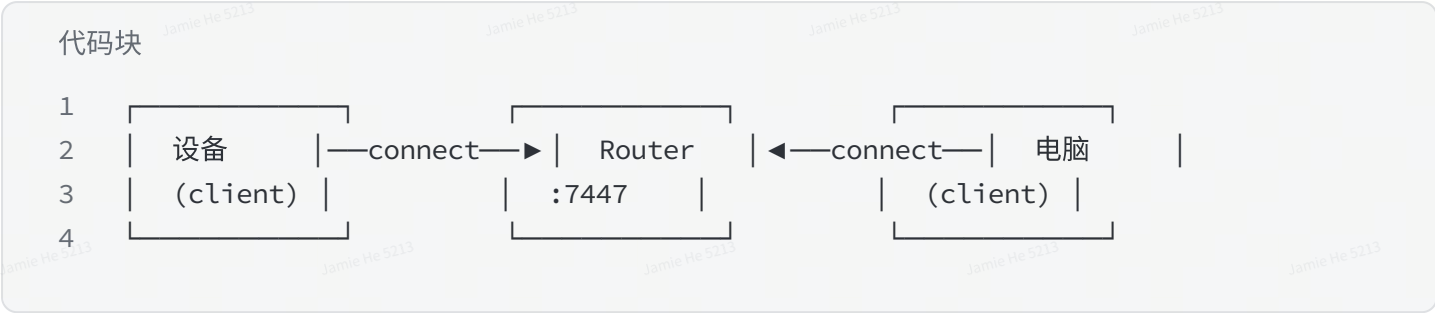
11、网络拓扑

场景 1：局域网直连（默认）



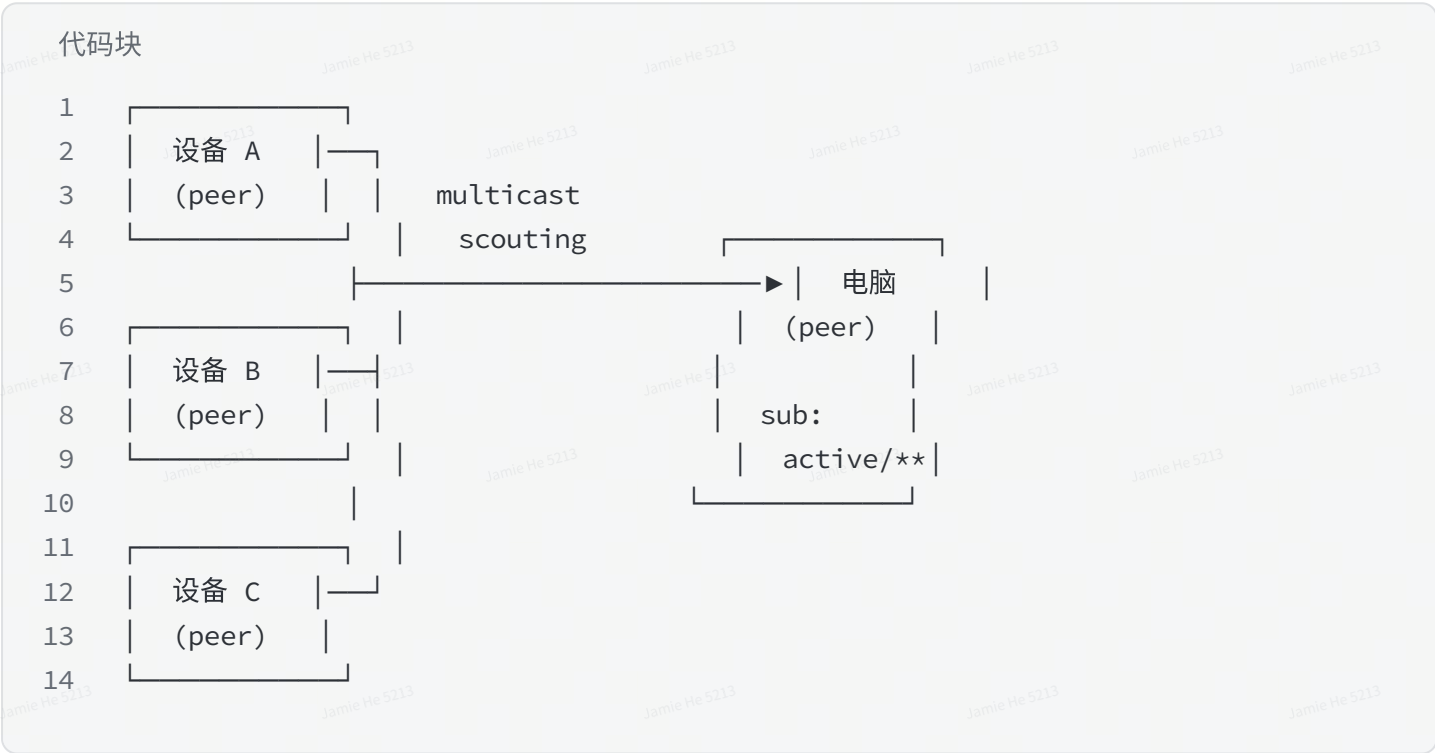
设备和电脑均配置 `mode: "peer"`，通过多播自动发现，无需配置 IP 端口。

场景 2：跨网段（通过 router）



设备和电脑配置 `mode: "client"`，`connect: ["tcp/router_ip:7447"]`。

场景 3：多设备局域网直连



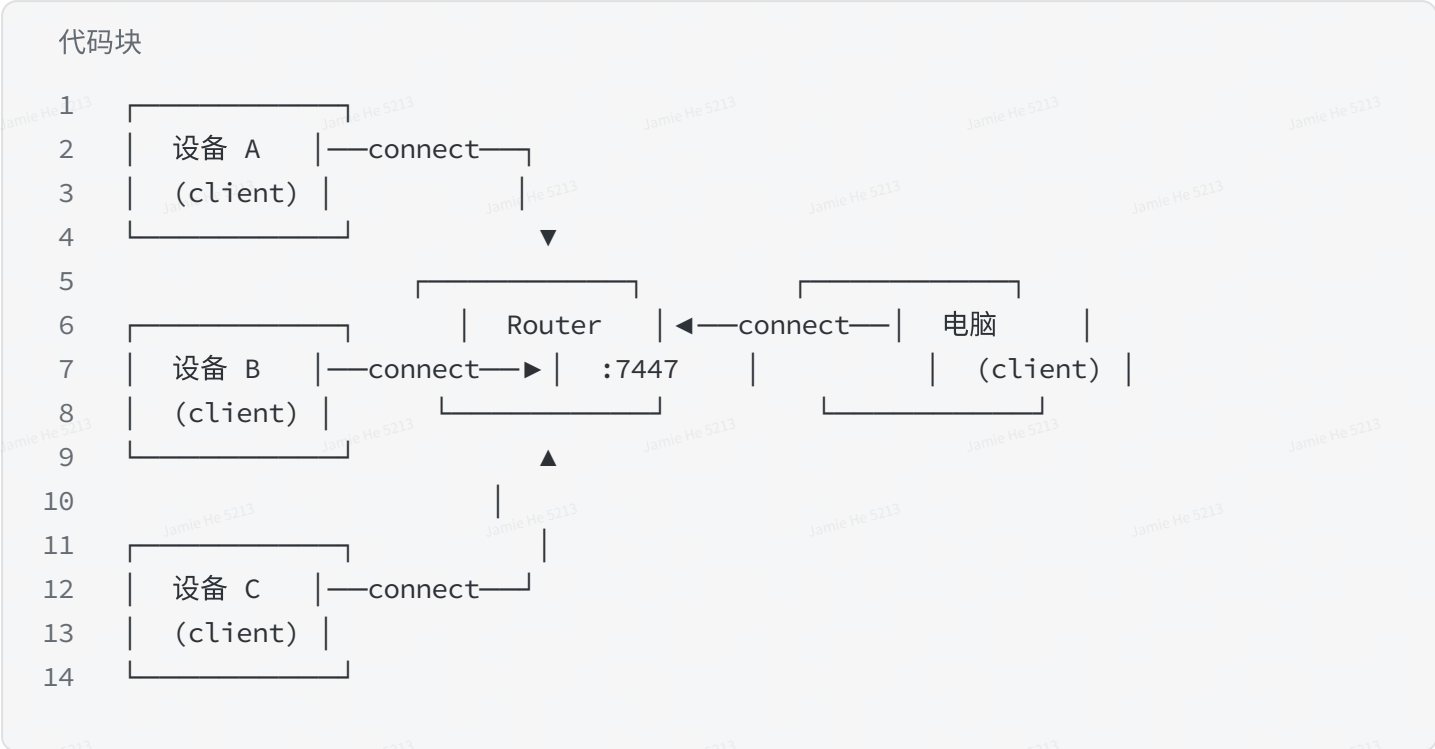
所有节点均配置 `mode: "peer"`，同一局域网内通过多播自动发现。电脑端通过 key expression 通配符区分设备：

代码块

```
1 // 订阅所有设备
2 session.declare_subscriber("active/**")
3
4 // 订阅特定设备
5 session.declare_subscriber("active/LX2601F10001/**")
6
7 // 订阅所有设备的点云
8 session.declare_subscriber("active/*/pointcloud")
```

每台设备独立 put 到各自的 key（如 `active/LX2601F10001/imu`、`active/LX2601F10002/imu`），订阅端无需任何额外配置即可接收所有设备数据。

场景 4：多设备通过 router 汇聚



适用场景：

- 设备分布在不同子网，无法多播发现
- 需要集中管理和监控多台设备
- 电脑端只需连接 router 即可获取所有设备数据

所有设备和电脑配置 `mode: "client"`，`connect: ["tcp/router_ip:7447"]`。Router 负责消息路由，各设备仍通过独立 key expression 区分。

多设备指令下发

电脑端向特定设备发送指令：

代码块

```
1 // 向设备 A 发送指令
2 session.get("active/LX2601F10001/cmd/reboot", payload)
3
4 // 向设备 B 发送指令
5 session.get("active/LX2601F10002/cmd/reboot", payload)
```

每台设备只响应自己 SN 对应的 queryable，不会收到其他设备的指令。

12、带宽预估（单台设备）

数据	频率	单帧预估大小	带宽/秒
点云	5 Hz	~400 KB (Protobuf varint 压缩)	2.0 MB/s
图像	10 Hz	~170 KB (1080p JPEG)	1.7 MB/s
IMU	50 Hz	~60 B	0.003 MB/s
总计			≈ 3.7 MB/s (30 Mbps)

注：Protobuf 的 varint 编码对点云坐标有较好的压缩效果（相比原始 float 格式），整体带宽有所降低。